Where in the World? Country-Level Geolocation from Street-View Imagery

Midterm Progress Report — Team GoodWillTuning

Tanner Temple
Davis College of Science & Engineering
Texas Christian University
Fort Worth, United States
TANNER.TEMPLE@tcu.edu

Esteban Hernandez
Davis College of Science & Engineering
Texas Christian University
Fort Worth, United States
E.HERNANDEZ1798@tcu.edu

Ethan Wong
Davis College of Science & Engineering
Texas Christian University
Fort Worth, United States
E.W.WONG@tcu.edu

I. Problem Statement and Changes from the Proposal

Given a single Google street-view photograph, we predict the country in which it was taken, similar to the game GeoGuessr with the additional constraints of not being able to pan the image or move around in street-view. The input is one RGB Google Street View image (640×640); the output is a probability distribution over country classes; the objective is to minimize cross-entropy loss.

The problem, dataset, evaluation metrics and planned neural models are unchanged from the approved proposal. Two refinements were made while implementing the baseline:

- **Scaled baseline.** The logistic regression baseline is run first as a *binary* classifier (two countries, e.g. France vs. Japan) and then on progressively larger subsets. This begins the planned controlled comparison over the number of classes at the smallest scale.
- **Native resolution.** The baseline uses the full 640×640 images without downsampling, so it sees the same information as the later models.

II. Dataset, Preprocessing, and Split Strategy

Source. We use *Streetview by Country* from Kaggle [3], downloaded with the Kaggle API and cached in Google Drive as a zip (about 7.9 GB) so each Colab session restores it in minutes. Images are stored in one folder per country, named by the two-letter country code (e.g. `FR`, `JP`); the folder name is the label.

Statistics. Table I summarizes the dataset as counted by `baseline_model.py --list-countries`.

TABLE I
Dataset statistics

Countries (classes)	111
Total images	107,269
Images per country (min / median / max)	110 / 1,000 / 1,005
Image resolution	640×640 RGB

Preprocessing (baseline). Each image is decoded to RGB and flattened to a vector of $640 \cdot 640 \cdot 3 = 1{,}228{,}800$ pixel values scaled to $[0, 1]$. Every feature is then standardized with the mean and standard deviation computed *on the training split only*. Because a full split does not fit in memory at this resolution, images are streamed from disk in mini-batches, and the training statistics are accumulated in one streaming pass.

Split strategy. For each selected country we shuffle its images with a fixed seed (42) and split them 70/15/15 into train, validation and test. The split is stratified by construction, so every class is represented in every split in the same proportion. The validation split is used for early stopping; the test split is evaluated once, after training finishes.

Checks for leakage. (i) Images are shuffled and partitioned within each country into non-overlapping train/validation/test slices, so each file belongs to exactly one split. At runtime, the training script verifies that the three sets of file paths are pairwise disjoint and that no image's MD5 hash appears in more than one split, which rules out the same photo being stored under two filenames; the run aborts if either check fails, and the result is logged with each experiment. (ii) Per-pixel normalization statistics (mean and standard deviation) are computed from training images only and then applied unchanged to the validation and test sets. (iii) Early stopping and checkpoint selection use validation loss only; the test set is evaluated exactly once, after training, and hyperparameters were not tuned on it. A remaining risk is spatial leakage: two photos taken a few meters apart could fall into different splits, letting a model match near-duplicate scenes instead of learning country-level cues. Hash matching also misses re-encoded or resized copies of the same image.

III. Models

Baseline: logistic regression. A single linear layer maps the 1,228,800 standardized pixels directly to class scores, with no hidden layers. With two countries it is binary logistic regression (one output, sigmoid, binary cross-entropy); As we learned, with $K > 2$ countries it is multinomial logistic regression which essentially functions as a one vs all approach

(K outputs, softmax, categorical cross-entropy). This baseline model is implemented in PyTorch and trained on a Colab GPU (Nvidia Tesla T4) with Adam optimization, a learning rate of $\alpha = 0.001$, a L2 weight decay of $\lambda = 10^{-4}$, batch size 64, up to 50 epochs, and early stopping with patience of 5 on validation loss, and the weights from the best validation epoch are kept. All settings are command-line arguments and are saved with each run's results.

Proposed model: convolutional neural network. We will try training a CNN from scratch as proposed before. Unlike logistic regression, a CNN can learn and extract features such as road markings, signs and vegetation, which are exactly what human GeoGuessr players use to guess. Our 1st iteration that we are about to implement follows the following architecture based on our limited knowlege of CNNs as of now. We will have 1 initial 7x7 convolutional layer, then 16 3x3 colvolutional layer operations then compressed and fed into 1 linear layer with K nodes for classification. We will use cross entropy loss, with the same optimizations as now. The architecture can be revised as we run more experiments.

IV. Preliminary Results

A. Training dynamics

Table II lists the loss and validation accuracy for every epoch. Training stopped after epoch 8 because validation loss did not improve on its epoch-3 minimum for five consecutive epochs.

TABLE II
PER-EPOCH TRAINING LOG. THE LOWEST VALIDATION LOSS IS IN BOLD.

Epoch	Train loss	Val loss	Val acc
1	48.92	43.95	0.710
2	20.71	23.13	0.707
3	8.87	**16.92**	0.700
4	4.89	29.63	0.683
5	8.15	18.95	0.677
6	4.25	20.19	0.700
7	5.21	20.23	0.763
8	3.53	21.73	0.730

Training loss dropped by more than an order of magnitude (48.9 to 3.5), but validation loss stopped improving after epoch 3 and finished well above it (21.7 versus 16.9). By epoch 8 the gap between validation and training loss was about 18 units, compared with a gap of only about 8 at epoch 3. This divergence is the signature of overfitting: the model keeps fitting the 1,404 training images while its behavior on unseen images does not improve. Validation accuracy tells the same story. It hovered between 0.68 and 0.76 for the whole run with no upward trend, so additional epochs were not buying better generalization. The training loss itself was also noisy and not monotonic (for example, it rose from 4.89 at epoch 4 to 8.15 at epoch 5), which suggests that the optimization was unstable at this step size on raw pixel features.

B. Test performance

Table III reports the test results. Overall top-1 accuracy is 0.730, which is 0.230 above chance. With 300 balanced test images, the standard error of a uniformly guessing classifier is about 0.029, so the result lies roughly eight standard errors above chance and is very unlikely to come from the particular split. Top-5 accuracy is trivially 1.000 with only two classes and carries no information here. Macro F1 (0.730) matches accuracy because the classes are balanced and the two per-class scores are nearly equal.

TABLE III
TEST-SET METRICS (300 IMAGES). OVERALL TOP-1 ACCURACY IS 0.730, MACRO F1 IS 0.730, AND CHANCE IS 0.500.

Class	Precision	Recall	F1	Support
FR	0.72	0.75	0.74	150
JP	0.74	0.71	0.72	150
Macro avg	0.73	0.73	0.73	300

C. Error analysis

Table IV shows the confusion matrix. Of the 300 test images, 81 (27%) were misclassified: 37 French images were labeled Japanese and 44 Japanese images were labeled French. The two error types are close in size, so the model does not collapse onto a single class. It does lean slightly toward French, predicting FR for 157 images and JP for 143 despite the even 150/150 split, which is why FR has higher recall and JP has higher precision.

TABLE IV
CONFUSION MATRIX ON THE TEST SET (ROWS ARE TRUE LABELS, COLUMNS ARE PREDICTED LABELS).

	Pred. FR	Pred. JP
True FR	113	37
True JP	44	106

D. Discussion

The baseline is learning, and works better than originally expected, predicting above random guessing chance of 0.500. However it won't be strong enough for our goal, being a very simple architecture, ran only on a few epochs, and roughly one image in four is still wrong on a task with only two classes. The expected difficulty grows sharply with the number of countries, because random chance at guessing falls from 0.500 at two classes to under 0.010 at 111. These results therefore serve mainly as a lower bound for the CNN models we plan to train, and prove that what we are doing is actually feasible.

V. Difficulties, Limitations, and Plan

Difficulties so far.

- **Memory and I/O.** At 640×640 a single image is 1.2M features; preloading even two countries needs several GB

of RAM. We moved to streaming from disk, so image decoding now dominates run time.

- **Overfitting risk.** The binary baseline has 1.2M weights but only about 1,400 training images, so it can memorize the training set. Weight decay and early stopping limit this.
- **Colab sessions.** Runtimes reset, so the dataset is cached in Google Drive and setup is scripted in `notebooks/init.ipynb`. There has also been difficulties with Google Colab and GitHub integration, as issues arise with permissions and collaborating with others
- **Python Notebook Issues** Using python notebooks along with GitHub collaboration can cause merge conflicts whenever they are heavily edited. We are currently circumventing this issue by writing all model code as normal python scripts in `scripts` and the notebook only downloads data and calls these python scripts on runtime

Limitations. Spatial leakage between nearby photos is a clear limitation as two images taken close together may prompt the model to memorize based upon certain pixels in certain positions rather than full generalizations.

Plan for the remaining work.

1) Run the baseline on 10, 50 and $\sim$100 countries with the same protocol (week of Oct. 20).
2) Implement and train the CNN from scratch (Oct. 27 – Nov. 10).
3) Run many different experiments to find best hyperparameters for this dataset and model (by Nov. 21)
4) Error analysis: confusion matrix of the most-confused country pairs, and representative successes and failures (by Nov. 21).
5) Bonus items: experiment tracking with Weights & Biases, Grad-CAM visualizations, and a demo (as time allows).
6) Final report (Dec. 2) and presentation (Dec. 7/9).

VI. Code

https://github.com/eggsbenedict287/tcu-cosc-50523-final-project-geoguessrbot

The README describes environment setup in Colab and the exact commands that reproduce each result above.

References

[1] T. Weyand, I. Kostrikov, and J. Philbin, "PlaNet - Photo Geolocation with Convolutional Neural Networks," in Proc. European Conference on Computer Vision (ECCV), 2016.

[2] J. Hays and A. A. Efros, "IM2GPS: Estimating Geographic Information from a Single Image," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2008.

[3] S. Shaw, "Streetview by country," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/sylshaw/streetview-by-country